



设备驱动通用配置指南

文档版本
发布日期

1.0
2020-07-20

版权所有 © 紫光展锐科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

紫光展锐科技有限公司



前言

概述

本文档介绍了 MOCOR 平台上设备驱动通用配置相关的数据结构及 API 接口，并举例说明设备驱动的配置。

读者对象

本文档主要适用于软件驱动开发人员。必须具备以下经验和技能：

- 熟悉 C。
- 熟悉 THREADX OS。

缩略语

缩略语	英文全名	中文解释
GPIO	General-purpose Input Output	通用输入/输出口
MMI	Man-Machine-Interface	人机接口

符号约定

符号	说明
 说明	用于突出重要/关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.0	2020-07-20	第一次正式发布

关键字

设备驱动、配置。

目 录

1 概述	1
1.1 设备驱动通用配置介绍	1
1.1.1 板级配置	1
1.1.2 芯片级配置	1
1.1.3 任务级配置	1
1.2 设备驱动配置框图	2
2 数据结构	3
2.1 宏	3
2.1.1 PIN 映射相关配置比特	3
2.1.2 DEF_TASK	3
2.2 枚举	4
2.2.1 GPIO_PROD_ID_E	4
2.2.2 GPIO_PROD_TYPE_E	5
2.2.3 thread_id_type	6
2.2.4 dyna_thread_id_type	7
2.2.5 SRV_ID_E	8
2.3 结构	9
2.3.1 PM_PINFUNC_T	9
2.3.2 GPIO_CFG_INFO_T	10
3 API 函数	11
3.1 PM_Init	11
3.2 GPIO_CustomizeInit	11
3.3 GPIO_SetDirection	12
3.4 GPIO_SetValue	12
3.5 _GPIO_PROD_SetVal	13
3.6 GPIO_PROD_RegGpio	13
3.7 SCI_CreateStaticThread	14
3.8 SCI_CreateAppThread	15
4 应用实例	18
4.1 Pinmap 配置	18
4.2 gpio_cfg	18
4.3 芯片级配置	19
4.4 静态 task 配置	19
4.5 线程入口函数	20

图目录

图 1-1 设备驱动配置框图	2
----------------------	---

表目录

表 2-1 工具列表	错误!未定义书签。
------------------	-----------

1 概述

1.1 设备驱动通用配置介绍

在 Threadx OS 添加一个设备驱动，需要结合芯片 SPEC 和板级电路图进行设备驱动配置。

设备驱动包含以下配置：

- 板级配置
- 芯片级配置
- 任务级配置

1.1.1 板级配置

板级配置包含 Pinmap 配置和 GPIO 配置：

- Pinmap 配置
Pinmap 配置是对硬件电路图结合芯片引脚寄存器对所涉及引脚进行统一初始化配置，主要在各项目的 `pinmap_cfg.c` 文件中。Pinmap 配置实例详见 4.1 Pinmap 配置。
- GPIO 配置
GPIO 配置针对设备驱动中的需求功能，在各项目的 `gpio_cfg.c` 文件中配置相应的 GPIO（详见 4.2 `gpio_cfg`）。若要对 GPIO 进行操作，如配置 GPIO I/O VALUE 等，需要在 `pm_gpio_default_map[]` 数组中注册，并使用 GPIO HAL 层接口。

Pinmap 配置初始化在 `PM_Init` 进行，CHIP 级初始化第一阶段；GPIO 配置初始化在 `GPIO_Init` 进行，CHIP 级初始化第三阶段。

1.1.2 芯片级配置

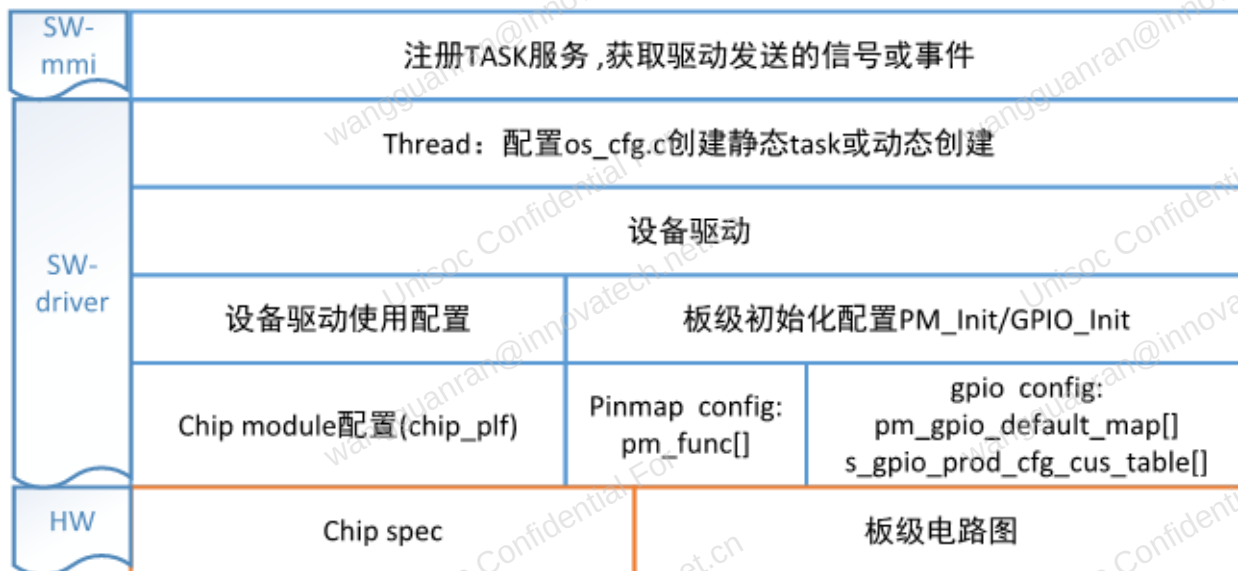
设备驱动需要使用芯片级驱动模块时，配置芯片驱动根据板级电路图设计进行客户化配置，MOCOR 平台的芯片级驱动配置分布在各个设备驱动使用时配置。可以参考各芯片驱动模块使用说明文档，代码在目录 `chip_drv\chip_plf` 下。芯片级配置应用实例详见 4.3 芯片级配置。

1.1.3 任务级配置

若设备驱动需要创建任务，则在 `os_cfg.c` 中进行配置创建静态任务或自行动态创建。使用 `SCI_CreateClientList()` 函数创建，通过 `SCI_SendEventToClient()` 函数发送信号/事件给 MMI，MMI 通过 `SCI_RegisterMsg()` 函数注册获取信号或事件。任务级配置应用实例详见 4.4 静态 task 配置以及 4.5 线程入口函数。

1.2 设备驱动配置框图

图1-1 设备驱动配置框图



如图 1-1 所示，设备驱动配置包括 SW-MMIO、SW-Driver 和 HW 三部分，模块驱动向上与 MMI 进行通讯，向下与硬件连接。为实现多个模块的统一配置，在 SW-Driver 中加了配置层，使得模块驱动可以兼容多个硬件，减少硬件间的耦合性，使得移植更加容易。

2 数据结构

2.1 宏

2.1.1 PIN 映射相关配置比特

IO 涉及驱动能力、功能复用、上下拉、输入输出等属性配置。

表2-1 PIN 映射相关配置比特

宏名	值	说明
BITS_PIN_DS(_x_)	((_x_) << 19) & (BIT_19 BIT_20 BIT_21 BIT_22)	驱动力配置
BIT_PIN_SLP_CM4	BIT_17	配置与相应的 sys 一起 sleep
BIT_PIN_SLP_WCN	BIT_16	配置与相应的 sys 一起 sleep
BIT_PIN_SLP_WTLCP	BIT_15	配置与相应的 sys 一起 sleep
BIT_PIN_SLP_PUBCP	BIT_14	配置与相应的 sys 一起 sleep
BIT_PIN_SLP_AP	BIT_13	配置与相应的 sys 一起 sleep
BIT_PIN_WPU_SEL	BIT_12	上拉电阻选择
BIT_PIN_SE	BIT_11	施密特触发器使能引脚，很少用到。
BIT_PIN_WPU	BIT_7	上拉
BIT_PIN_WPD	BIT_6	下拉
BITS_PIN_AF(_x_)	((_x_) << 4) & (BIT_4 BIT_5)	复用功能选择
BIT_PIN_SLP_WPU	BIT_3	Sleep 上拉
BIT_PIN_SLP_WPD	BIT_2	Sleep 下拉
BIT_PIN_SLP_IE	BIT_1	深度睡眠下输入使能
BIT_PIN_SLP_OE	BIT_0	深度睡眠下输出使能

2.1.2 DEF_TASK

使用 DEF_TASK 宏定义静态创建 task。

```
#define DEF_TASK(__name, __prio, __stack_size, __queue_num) \
{ \
    void* __name##_STACK_ADDR = (void*)SCI_ALLOC_CONST((( __stack_size >> 2) << 2) + 3); \
    char* __name##_QUEUE_ADDR = (char*)SCI_ALLOC_CONST( __queue_num * SCI_QUEUE_ITEM_SIZE \
    * 4 + 3); \
    extern void thread_entry_## __name(uint32, void*); \
    SCI_ASSERT((__name##_STACK_ADDR != SCI_NULL) && (__name##_QUEUE_ADDR != SCI_NULL)); \
    __name##_STACK_ADDR = (void*)((((uint32) __name##_STACK_ADDR + 3) >> 2) << 2); \
    __name##_QUEUE_ADDR = (char*)((((uint32) __name##_QUEUE_ADDR + 3) >> 2) << 2); \
    SCI_CreateStaticThread( __name, "T_"# __name, thread_entry_## __name, 0, 0, \
    __name##_STACK_ADDR, __stack_size, __prio, SCI_PREEMPT, "Q_"# __name, \
    __name##_QUEUE_ADDR, __queue_num, SCI_AUTO_START); \
}
```

2.2 枚举

2.2.1 GPIO_PROD_ID_E

【功能】

GPIO 配置以 LOGICAL ID 格式在 gpio_cfg.c 配置文件中的数组 s_gpio_prod_cfg_cus_table[]里配置。

【定义】

```
typedef enum
{
    GPIO_PROD_RF_RESET_ID,
    GPIO_PROD_DSP_RESET_ID,
    GPIO_PROD_DSP_HL0D_ID,
    GPIO_PROD_LCD_BL_EN_ID,
    GPIO_PROD_LCD_BL_BRIGHTNESS_ID,
    GPIO_PROD_LCD_RST_ID,
    GPIO_PROD_KEYPAD_BL_ID,
    GPIO_PROD_CHARGE_PLUG_DETECT_ID,
    GPIO_PROD_VIBRATOR_EN_ID,
    GPIO_PROD_FLASH_EN_ID,
    GPIO_PROD_FLIP_ON_ID,
    GPIO_PROD_SLIDE_ON_ID,
    .....,
    GPIO_PROD_HEADSET_DETECT_ID,
    GPIO_PROD_HEADSET_BUTTON_ID,
    GPIO_PROD_TFXL_DETECT_ID,
```

```
GPIO_PROD_POWER_KEY_DETECT_ID,
GPIO_PROD_USB_DETECT_ID,
GPIO_PROD_SENSOR_POWER_ID,
GPIO_PROD_SENSOR_PWDN_ID,
GPIO_PROD_SENSOR_PWDN_FRONT_ID,
GPIO_PROD_SENSOR_RESET_ID,
GPIO_PROD_SENSOR_RESET_FRONT_ID,
.....

GPIO_PROD_TP_INT_ID,
GPIO_PROD_TP_WAKE_ID,
#if defined(MOTION_SENSOR_TYPE_TS1003)
GPIO_PROD_GSENSOR_E1_ID,
GPIO_PROD_GSENSOR_E2_ID,
#endif
.....

GPIO_PROD_I2C_SDA1,
GPIO_PROD_I2C_SCL1,
GPIO_PROD_I2C_SDA2,
GPIO_PROD_I2C_SCL2,
GPIO_PROD_TCXO_DCXO_ADATION_ID,
GPIO_PROD_ID_MAX
} GPIO_PROD_ID_E
```

【参数说明】

枚举值	说明
GPIO_PROD_XXX_ID	Logical id 必须小于 GPIO_PROD_ID_MAX。
GPIO_PROD_ID_MAX	Max value, if logical id is beyond the value, system will assert.

2.2.2 GPIO_PROD_TYPE_E

【功能】

定义 GPIO 配置中 GPIO 的类型。

【定义】

```
typedef enum
{
    GPIO_PROD_TYPE_BB0 = 0x00, // from Baseband
    GPIO_PROD_TYPE_EXT,        // from Extend Gpio ic
    GPIO_PROD_TYPE_EIC_DBNC,
```

```
GPIO_PROD_TYPE_EICA_DBNC,
GPIO_PROD_TYPE_MAX
}GPIO_PROD_TYPE_E;
```

【参数说明】

枚举值	说明
GPIO_PROD_TYPE_BB0	基于芯片 BB
GPIO_PROD_TYPE_EXT	外部 IC 的 GPIO
GPIO_PROD_TYPE_EIC_DBNC	一种 EIC 中断模式，具有 de-bounce（1ms~4s），少于 1ms 则不产生中断。
GPIO_PROD_TYPE_EICA_DBNC	具有唤醒功能的 de-bounce 中断模式

2.2.3 thread_id_type

【功能】

静态 thread id。

【定义】

```
typedef enum
{
/* Here is for Refence design. From 0 to 11*/
    CM                = 0,          // Channel manager thread defination.
    DIAG              = 1,          // Diagnostic thread defination.
    KPDSVR            = 2,
    RESERVED3         = 3,
    AUDIO             = 4,          // AUDIO
    VIDEO             = 5,          // VIDEO
    EFS               = 6,
    CHR_SVR           = 7,
    UDISK             = 8,
    TIMESVR           = 9,
    FORBIDDEN_USE10   = 10,         // Reserved for MIDI(for D platform) and MP3
    P_TPP             = 11,         // Touch Panel Task
/* Here is for Layer1 design. From 12 to 14*/
    P_LAYER1          = 12,
    TESTDSPTOL1       = 13,         //just for test
    TESTPSTOL1        = 14,         //just for test
/* Here is for simat design. From 15 to 17*/
```

```

P_SIMAT_IMPL      = 15,
P_SIMAT_DUM       = 16,
P_SIMAT           = 17,
/* Here is for Protocol Stack design. From 18 to 20*/
P_RRA             = 18,
P_MN              = 19,
RECORD            = 20,
P_APP             = 21,          // used for APP
/* Here is for SIM design. From 22 to 23*/
P_SIM             = 22,
P_SIM_WIN_DUMMY   = 23,
P_ATC             = 24,
FFS              = 25,          // RESERVED25 = 25
MPEG              = 26,
CUSTOMIZED_STATIC_TASKS // 12 tasks id
// Add your tasks id here.
// But all static tasks MUST be less than SYSTEM_STATIC_THREAD_MAX_NUM !!!
STL_COMPACTION    = 39,
P_CPANEL          = 40,
P_PSDummy         = 41,
P_NVSMS           = 42,
.....
MAX_BLOCK_NUM = USER_CUSTOMIZE_TASK_ID_START
} thread_id_type;

```

【参数说明】

枚举值	说明值
所有枚举值	静态线程 logical id，取值小于 SYSTEM_STATIC_THREAD_MAX_NUM。

2.2.4 dyna_thread_id_type

【功能】

动态线程 Id 类型

【定义】

```

typedef enum
{
    STARUP_DYNA_TASK_ID = USER_CUSTOMIZE_TASK_ID_END + 1,
    CUSTOMIZED_DYNAMIC_TASKS

```



```
P_TL,
P_SMS,
P_NETWORK,
MSG_BUF,
SIO,
P_SN,
P_DSP,
GPRS_LOG_MSG_ID = 0x90
}dyna_thread_id_type;
```

【参数说明】

枚举值	说明
所有枚举值	动态线程 logical id 取值范围 [USER_CUSTOMIZE_TASK_ID_START, USER_CUSTOMIZE_TASK_ID_END]

2.2.5 SRV_ID_E

【功能】

MMI 层注册服务的各种类型。

【定义】

```
typedef enum
{
    FILE_SERVICE = 1,           // File service, Reserved
    AUDIO_SERVICE,             // Audio service
    KEYPAD_SERVICE,            // Keypad service
    USB_SERVICE,                // Udisk service
    UPM_SERVICE=USB_SERVICE,
    CHR_SERVICE,                // Charge service
    TIME_SERVICE,              // RTC service
    INTERNAL_USER_SERVICE = 7,  // For MN and such service
    INTERNAL_USER_SERVICE8,
    INTERNAL_USER_SERVICE9,
    .....
    INTERNAL_USER_SERVICE63,
    SYSTEM_SERVICE = 64,        // 64~127 for system
```

```
.....
DBAT_SERVICE = 99,
TP_SERVICE = 100,          // For Touchpanel service
ISP_SERVICE,
EXTERNAL_USER_SERVICE = 128,  // 128~255 for external user
EXTERNAL_USER_SERVICE129,
.....
EXTERNAL_USER_SERVICE255,
MAX_SERVICE = 256
} SRV_ID_E;
```

【参数说明】

枚举值	说明
所有 MMI 层注册服务的枚举值	MMI 层注册服务相应的 logical id
MAX_SERVICE	服务 logical id 不能超过此值

2.3 结构

2.3.1 PM_PINFUNC_T

【功能】

Pinmap 配置以 PM_PINFUNC_T 结构进行配置。Value 值为宏定义中 IO 属性。

【定义】

```
typedef struct PM_PINFUNC_tag
{
    uint32 addr;
    uint32 value;
} PM_PINFUNC_T;
```

【参数说明】

参数	说明
addr	Reg address for PIN
value	上述宏定义中比特位的各值组成；含义在不同寄存器地址但 BIT 位配置功能相同

2.3.2 GPIO_CFG_INFO_T

【功能】

GPIO 配置相关信息，包括 GPIO 的 ID、类型、有效电平等。

【定义】

```
typedef struct
{
    GPIO_PROD_ID_E      gpio_id;
    GPIO_PROD_TYPE_E    gpio_type;
    uint16              valid_level;
    uint16              gpio_num;
    GPIO_CB             gpio_cb_fun;
}GPIO_CFG_INFO_T, *GPIO_CFG_INFO_T_PTR;
```

【参数说明】

参数	说明
gpio_id	gpio logical id
gpio_type	gpio type
valid_level	1: High level valid 0: low level valid
gpio_num	gpio real number
gpio_cb_fun	Callback function

3 API 函数

3.1 PM_Init

【函数功能】

对 pin 的 map 配置表 pm_func[] 进行初始化，包括 A-DIE/D-DIE 的 pin 配置。该初始化配置会在 CHIP 的第一阶段完成。

【函数原型】

```
PUBLIC void PM_Init (void);
```

【参数说明】

无

【返回值】

无

3.2 GPIO_CustomizeInit

【函数功能】

GPIO 客户化配置初始化函数，包括对配置数组 s_gpio_prod_cfg_cus_table 的初始化。数组中的索引来自 GPIO_PROD_ID_E 枚举。该 GPIO 客户化配置在 chip base 初始化基本完成后的第三阶段进行，包含在 GPIO_Init() 函数中。

【函数原型】

```
PUBLIC void GPIO_CustomizeInit(void);
```

【参数说明】

无

【返回值】

无

3.3 GPIO_SetDirection

【函数功能】

操作 GPIO 的输入输出功能

【函数原型】

```
PUBLIC void GPIO_SetDirection (uint32 gpio_id, BOOLEAN directions);
```

【参数说明】

参数	说明
Gpio_id	Real gpio id
directions	• 1: output • 0: input

【返回值】

无

3.4 GPIO_SetValue

【函数功能】

设置 GPIO 口的值。

【函数原型】

```
PUBLIC void GPIO_SetValue (uint32 gpio_id, BOOLEAN b_on);
```

【参数说明】

参数	说明
gpio_id	Real gpio id
b_on	• 1: on • 0: off

【返回值】

无

3.5 _GPIO_PROD_SetVal

【函数功能】

通过 ID 调用 callback 函数，传入不同布尔值。相关 ID 的配置在 `s_gpio_prod_cfg_cus_table[]` 数组中定义。

【函数原型】

```
BOOLEAN _GPIO_PROD_SetVal (GPIO_PROD_ID_E id, BOOLEAN value);
```

【参数说明】

参数	说明
id	查看 GPIO_PROD_ID_E 枚举类型
value	TRUE 或 FALSE

【返回值】

- 0: Success
- 1: Fail

3.6 GPIO_PROD_RegGpio

【函数功能】

通过 ID 注册 GPIO 功能，相关 ID 配置在 `s_gpio_prod_cfg_cus_table[]` 数组中定义。

【函数原型】

```
BOOLEAN GPIO_PROD_RegGpio (
    GPIO_PROD_ID_E    id,
    BOOLEAN            direction,
    BOOLEAN            default_value,
    BOOLEAN            shaking_en,
    uint32             shaking_interval,
    GPIO_PROD_CALLBACK gpio_callback_fun
);
```

【参数说明】

参数	说明
id	查看 GPIO_PROD_ID_E 枚举类型

参数	说明
direction	在 gpio_type 类型为 GPIO_PROD_TYPE_EXT 时使用 - FALSE:input - TRUE:output
default_value	在 gpio_type 类型为 GPIO_PROD_TYPE_EXT 时使用，设置默认初始值。
shaking_en	去抖使能位 - TRUE：使能 - FALSE：不使用该功能
shaking_interval	去抖时间(单位 ms)
gpio_callback_fun	Callback 函数

【返回值】

- 0: Success
- 1: Fail

3.7 SCI_CreateStaticThread

【函数功能】

在 os_cfg.c 中静态创建线程。

【函数原型】

```

PUBLIC uint32 SCI_CreateStaticThread(
    BLOCK_ID    thread_id,
    const char  *thread_name,
    void        (*entry)(uint32, void*),
    uint32      argc,
    void        *argv,
    void        *stack_start,
    uint32      stack_size,
    uint32      priority,
    uint32      preempt,
    const char  *queue_name,
    void        *queue_start,
    uint32      queue_num,
    uint32      auto_start
);
  
```

【参数说明】

参数	说明
thread_id	不能大于 SCI_MAX_STATIC_THREAD_ID
thread_name	线程名
Entry()	线程入口函数
argc	线程入口函数参数一
argv	线程入口函数参数二
stack_start	栈起始地址，DEF_TASK 会初始化
stack_size	栈大小
priority	线程优先级
preempt	抢占使能： - SCI_NO_PREEMPT - 不使能 - SCI_PREEMPT - 使能（默认使能）
queue_name	队列名
queue_start	队列起始值指针，DEF_TASK 会初始化
queue_num	队列接收信息数量
auto_start	是否自动开始： - SCI_DONT_START - 线程手动使能 - SCI_AUTO_START - 线程创建后自动开始运行（默认自动开始）

【返回值】

SCI_SUCCESS: 成功创建线程

3.8 SCI_CreateAppThread

【函数功能】

动态创建线程

【函数原型】

```
PUBLIC BLOCK_ID SCI_CreateAppThread( // If successful, returns the thread ID
                                     // of the new created thread,
```

```

// else returns SCI_INVALID_BLOCK_ID
const char *thread_name, // Name string of the thread
const char *queue_name, // Queue name string of the thread
void (*entry)(uint32, void*), // Entry function of the thread
uint32 argc, // First parameter for entry function,
void *argv, // Second parameter for entry function,
uint32 stack_size, // Size of the thread stack in bytes
uint32 queue_num, // Number of messages which can be enqueued
uint32 priority, // Priority of the thread.
uint32 preempt, // Indicates if the thread is preemptable.
uint32 auto_start // Specifies whether the thread starts
// immediately or stays in a pure suspended
// state. Legal options are SCI_AUTO_START
// and SCI_DONT_START.
);

```

【参数说明】

参数	说明
thread_name	线程名
queue_name	队列名
Entry()	线程入口函数
argc	线程入口函数参数一
argv	线程入口函数参数二
stack_size	栈大小
queue_num	队列接收信息数量
priority	线程优先级
preempt	抢占使能： • SCI_NO_PREEMPT - 不使能 • SCI_PREEMPT - 使能
auto_start	是否自动开始： • SCI_DONT_START - 线程手动使能 • SCI_AUTO_START - 线程创建后自动开始运行

【返回值】

- Thread_id: 成功创建线程

- SCI_INVALID_BLOCK_ID: 创建线程失败

4 应用实例

下面分别对板级配置，芯片级配置和 **task** 分别给出了部分代码。方便大家查阅

4.1 Pinmap 配置

```
const PM_PINFUNC_T pm_func[] =
{
    {REG_PIN_IIS1DI, BITS_PIN_AF(3)},
    {REG_MISC_PIN_IIS1DI, BITS_PIN_DS(1)|BIT_PIN_NULL|BIT_PIN_WPD|
    BIT_PIN_SLP_AP|BIT_PIN_SLP_WPD|BIT_PIN_SLP_Z},
    {REG_PIN_IIS1DO, BITS_PIN_AF(3)},
    {REG_MISC_PIN_IIS1DO, BITS_PIN_DS(1)|BIT_PIN_NULL|BIT_PIN_WPD|
    BIT_PIN_SLP_AP|BIT_PIN_SLP_WPD|BIT_PIN_SLP_Z},
    {REG_PIN_IIS1CLK, BITS_PIN_AF(3)},
    {REG_MISC_PIN_IIS1CLK, BITS_PIN_DS(1)|BIT_PIN_NULL|BIT_PIN_WPD|
    BIT_PIN_SLP_AP|BIT_PIN_SLP_WPD|BIT_PIN_SLP_Z},
    {REG_PIN_IIS1LRCK, BITS_PIN_AF(3)},
}
```

4.2 gpio_cfg

```
LOCAL const GPIO_CFG_INFO_T s_gpio_prod_cfg_cus_table[] =
{
    {GPIO_PROD_LCD_BL_EN_ID, GPIO_PROD_TYPE_MAX,
    GPIO_PROD_HIGH_LEVEL,GPIO_PROD_NUM_INVALID, _GPIO_OpenLCMBackLight},
    {GPIO_PROD_KEYPAD_BL_ID, GPIO_PROD_TYPE_MAX,
    GPIO_PROD_HIGH_LEVEL,GPIO_PROD_NUM_INVALID, _GPIO_OpenKeyPadBackLight},
    {GPIO_PROD_VIBRATOR_EN_ID, GPIO_PROD_TYPE_MAX,
    GPIO_PROD_HIGH_LEVEL,GPIO_PROD_NUM_INVALID, _GPIO_OpenVibrator},
    {GPIO_PROD_HEADSET_DETECT_ID, GPIO_PROD_TYPE_EICA_DBNC,
    GPIO_PROD_HIGH_LEVEL,EICA_AUD_HEAD_INSERT_ALL,PNULL},
    {GPIO_PROD_HEADSET_BUTTON_ID, GPIO_PROD_TYPE_EICA_DBNC,
    GPIO_PROD_HIGH_LEVEL,EICA_AUD_HEAD_BUTTON,PNULL},
    {GPIO_PROD_SPEAKER_PA_EN_ID, GPIO_PROD_TYPE_MAX,
    GPIO_PROD_HIGH_LEVEL,GPIO_PROD_NUM_INVALID, _GPIO_OpenPA},
    {GPIO_PROD_CHARGE_PLUG_DETECT_ID, GPIO_PROD_TYPE_EICA_DBNC, GPIO_PROD_HIGH_LEVEL,
    EICA_CHGR_INT, PNULL},
    {GPIO_PROD_POWER_KEY_ID, GPIO_PROD_TYPE_EICA_DBNC, GPIO_PROD_LOW_LEVEL, EICA_PBINT,
```

```
PNULL},
{GPIO_PROD_MENUCANCEL_KEY_ID, GPIO_PROD_TYPE_EICA_DBNC, GPIO_PROD_HIGH_LEVEL,
EICA_RSTN, PNULL},
}
```

4.3 芯片级配置

在设备驱动需要使用芯片驱动模块时，配置芯片驱动可以参考各驱动模块说明文档。

下面是 Touchpanel 使用 I2C 的一段参考代码。

TPC_GetCtrlPort 获得使 I2C 的逻辑 ID，调用 I2C HAL 层接口函数 I2C_Open()进行初始化。

```
LOCAL uint32 Zinitix_Get_I2c_Port(struct i2c_client *i2c_dev)
{
    i2c_dev->index = (uint32) TPC_GetCtrlPort();
    if (_Get_Zinitix_I2C_Status == TP_MULTI_ERROR) {
        CTP_Debug("Get the i2c port failed!");
        return TP_MULTI_ERROR;
    }

    if ( I2C_Open(i2c_dev)) {
        CTP_Debug("Open the i2c port failed!");
        return TP_MULTI_ERROR;
    } else {
        CTP_Debug("Open the i2c port Suceess!");
        return TP_MULTI_SUCCESS;
    }
}
```

4.4 静态 task 配置

```
STATIC_TASK_DEF_BEGIN(Normal)
// DEF_TASK(<THREAD_NAME>, <PRIORITY>, <STACK_SIZE>, <QUEUE_NUM>)
#ifndef WIN32
    DEF_TASK( CM, PRI_CM, (4096), 50 )
    DEF_TASK( DIAG, PRI_DIAG, (4096), 20 )
#endif
    DEF_TASK( KPDSVR, PRI_KPDSVR, (1500), 40 )
    DEF_TASK( AUDIO, PRI_AUDIO, (1024 * 4), 100 )
STATIC_TASK_DEF_END
```

4.5 线程入口函数

```
THREAD_ENTRY (KPDSVR)
{
    SVR_SIG_T *KPD_sig_ptr    = SCI_NULL;
    SCI_CreateClientList (KPD_SERVICE, (KPD_MSG_MAX_NUM & 0x0ff),
        (REG_CALLBACK) KPDSVR_Callback);
    HAL_InitKPD ();
    While (SCI_TRUE)
    {
        //Receive a signal.
        KPD_sig_ptr = (XXXSVR_SIG_T *) SCI_GetSignal (KPD_SVR);
        SCI_ASSERT (KPD_sig_ptr);
        //Send message to client
        SCI_SendEventToClient (KPD_SERVICE,
            KPD_sig_ptr->SignalCode, (void *) KPD_sig_ptr-> KPD);
        SCI_FREE (sig);
    }
}
```